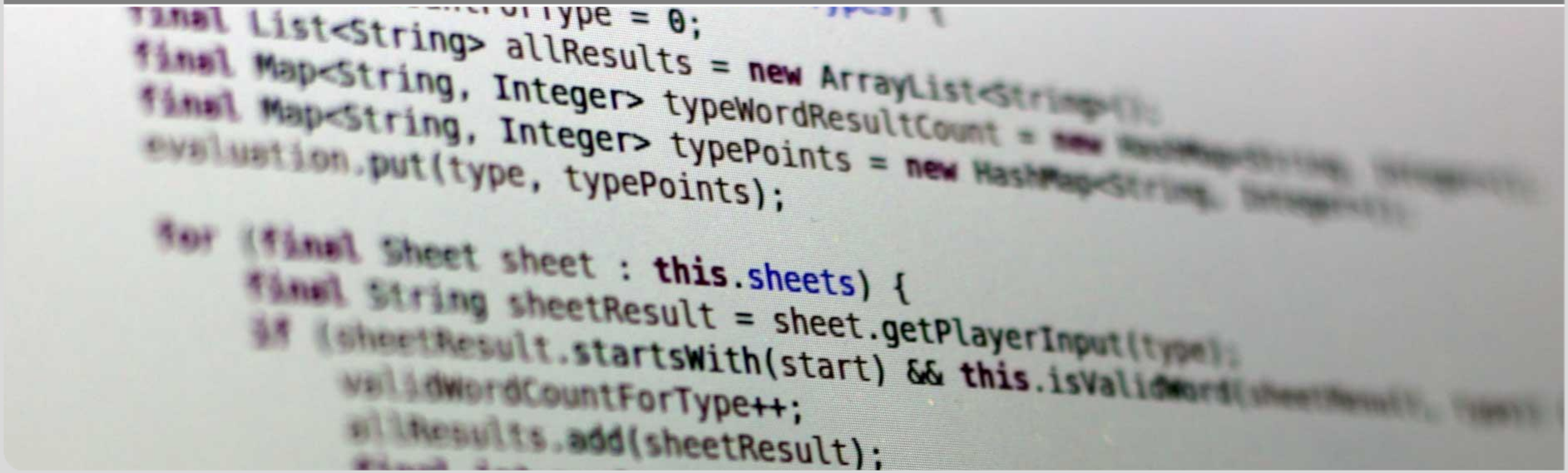
Karlsruher Institut für Technologie

DHBW
Duale Hochschule
Baden-Württemberg

Programmieren II

Apache Maven

Institut für Automation und angewandte Informatik



Agenda

- Einführung in Apache Maven
  - Was ist Apache Maven?
  - Verwendung
  - Einbindung in IDE



Was ist Apache Maven?

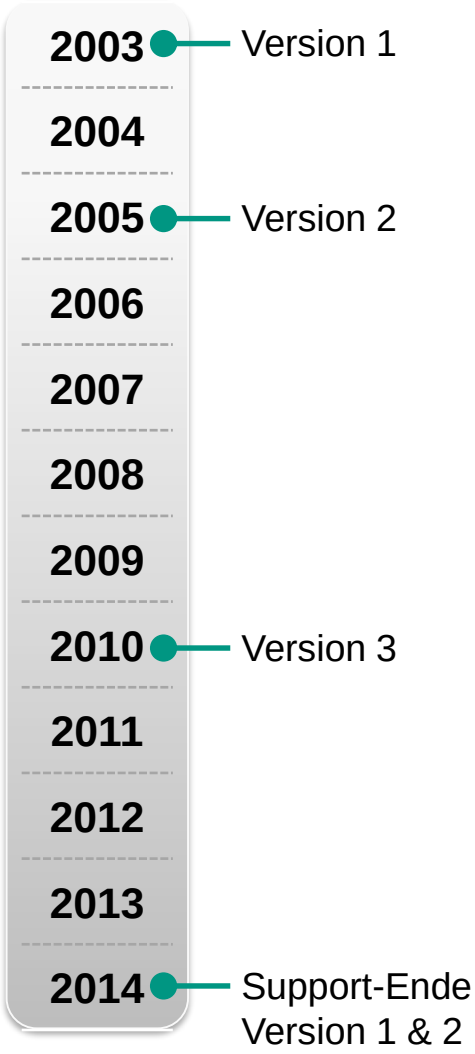
- „Maven ist ein Build-Management-Tool der Apache Software Foundation und basiert auf Java. Mit ihm kann man insbesondere Java-Programme standardisiert erstellen und verwalten.“
  (Quelle: https://de.wikipedia.org/wiki/Apache_Maven)

- Kommandozeilen-basiertes Tool

- Ziele
  - Vereinfachung und Vereinheitlichung des Build-Prozesses
  - Bereitstellung von Projektinformationen
  - Förderung von „Best Practices“
  - Transparentes Einbinden zusätzlicher Features mittels Plugins

maven

Entstehung



- Maven 1
  - Relativ hektische Umsetzung, daher einige Schwächen
- Maven 2
  - Konzept komplett überarbeitet
  - Fehler von Maven 1 korrigiert
  - Inkompatibel zu Maven 1
- Maven 3
  - Kompatibel zu Maven 2
  - Ziel war Verbesserung der Benutzbarkeit und Stabilität

Vereinfachung und Vereinheitlichung des Build-Prozesses

- Die Details verschiedener Build-Mechanismen werden über vereinfachte Wrapper bereitgestellt
  - Struktur der Wrapper meist ähnlich, trotz sehr unterschiedlicher Tools die darunter angesprochen werden

- Projektbeschreibung mittels “Project Object Model” (POM)
  - Beschreibung der Abhängigkeiten
  - Maven-Projekte “funktionieren” alle gleich
  - Zusatzfunktionalität wird über Plugins standardisiert eingebunden

Bereitstellung von Projektinformationen

- Standardisierte Informationen über Projekt
  - Grobe Zugehörigkeit zu Projektgruppe („groupId“)
  - Konkretes Projekt innerhalb der Gruppe („artifactId“)
  - Versionsnummer (inkl. Markierung von Entwicklungsversionen)
  - Optional möglich
    - Ausführlicher Name des Projekts
    - Beschreibungstext für das Projekt
- Klare Definition verwendeter Software durch zentral verwaltete Abhängigkeiten
- Direkte Einbindung von SCM-Systemen (Git, SVN, ..) vorgesehen
- Möglichkeit zur automatischen Generierung von Test-Reports

maven

Förderung von „Best Practices“

- Standardisierte Projektstruktur („Ordner-Struktur“)
  - Nicht mehr abhängig von der jeweiligen IDE
- Best Practices werden gefördert, jedoch nicht erzwungen
- Beispiel: Unit-Tests
  - Tests sind zentraler Bestandteil des Maven-Entwicklungszyklus‘
  - Test-Klassen werden in separatem, parallel zu den „normalen“ Klassen liegenden Quellcode-Baum erwartet
  - Namenskonventionen müssen eingehalten werden, damit Tests gefunden werden
  - Keine Custom-Umgebung für Tests
    - Test-Klassen müssen Test-Umgebung selbst initialisieren

Transparentes Einbinden zusätzlicher Features

- Einfach gehaltenes Plugin-System für Erweiterung der Funktionalität
- Sauber geschriebene Maven-Plugins funktionieren für alle Software-Projekte
  - „Früher“: geskriptete Sonderlösungen meist für konkretes Projekt
- Plugin-Update kann Verbesserungen für alle verwendenden Projekte bereitstellen

Abhängigkeitsverwaltung

- Wenn Software-Abhängigkeiten automatisch verwaltet werden, müssen diese „irgendwo“ verfügbar sein
- Maven verwendet hierfür Repositories
- Es gibt 2 Arten von Repositories
  - Lokal: Ordner auf dem Rechner mit dem Maven-Projekt. Verwaltet eigene Projekt-Artefakte und cachet verwendete Drittsoftware
  - Remote: Server, der bestimmten „Katalog“ von Software bereitstellt, die in Maven-Projekten genutzt werden kann

Abhängigkeitsverwaltung II

- Abhängigkeiten werden *beschrieben*, nicht heruntergeladen und in das Projekt kopiert
  - Bibliotheken liegen nicht mehr in „Binär“-Form innerhalb des Software-Projekts
  - **Das Projekt kann vollständig in ein Versionskontrollsystem geladen werden, ohne Fremd-Software dort mitzuverwalten**

Lebenszyklus eines Maven Projekts

- Maven bildet einzelne Schritte des Projekts in einem Lebenszyklus ab

- Häufig ausreichend: Standard-Lebenszyklus
  - Nicht jedes Software-Projekt muss jede der Phasen durchlaufen

Phase	Beschreibung
`archetype`	Erstellung des Software-Projekts, ggf. auf Basis eines zuvor definierten Templates
`validate`	Überprüfung der Projektstruktur auf Gültigkeit und Vollständigkeit
`compile`	Übersetzung des Quellcodes (z.B. `.java` → `.class`)

Lebenszyklus eines Maven Projekts II

Phase	Beschreibung
`test`	Ausführung der Unit-Tests aus dem parallelen Quellcode-Baum mit passendem Test-Framework (z.B. JUnit)
`package`	Die übersetzten Dateien werden zum fertigen „Produkt“ verpackt, bspw. ein JAR-Archiv. Hier werden ggf. auch Dateien eingefügt die nicht kompilierbar sind (Bilder, XML-Dateien, …)
`integration-test`	Fertiges Produkt kann in eine andere Umgebung übertragen werden und dort mit anderen Komponenten integriert getestet werden
`verify`	Überprüfung der Struktur des erzeugten Softwarepakets, ggf. Überprüfung von Qualitätskriterien
`install`	Installation im lokalen Maven-Repository
`deploy`	Installation in einem entfernten Maven-Repository

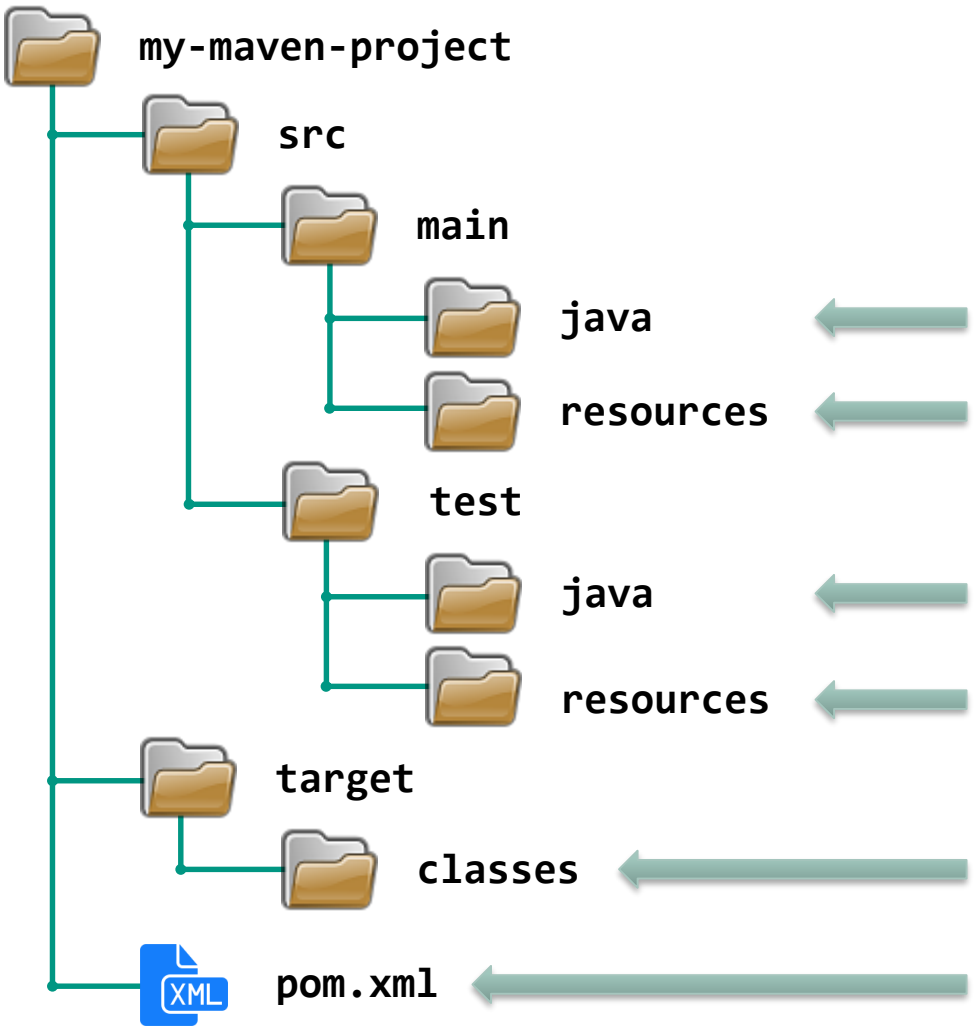
Voraussetzungen

- Maven basiert auf Java und soll Software erzeugen
  - ➔ installiertes JDK nötig
  - ➔ Umgebungsvariable `$JAVA_HOME` muss gesetzt sein

- Variante 1: Maven herunterladen und installieren
  - Download von http://maven.apache.org
  - Installation besteht aus Entpacken & Anpassen der Umgebungsvariable $PATH

- Variante 2: IDE-Integration
  - m2e-Plugin für Eclipse (in neueren Eclipse-Versionen bereits enthalten)
  - NetBeans & IntelliJ bringen ebenfalls bereits Plugins mit

Projektstruktur



- **java** ← Java-Packages & -Klassen
- **resources** ← Nicht übersetzbare Dateien (Bilder, …)
- **java** ← Java-Packages & -Klassen **für Unit-Tests**
- **resources** ← Nicht übersetzbare Dateien (Bilder, …) **für Unit-Tests**
- **classes** ← Erzeugte, kompilierte Dateien (`.class`)
- **pom.xml** ← „*Project Object Model*“-Konfigurationsdatei

maven

Project Object Model – `pom.xml`

- Zentrale Konfigurationsdatei für Maven-Projekt
- Definiert und instrumentiert zuvor genannte Funktionen
  - Projektinformationen
  - Abhängigkeiten zu anderen Software-Bibliotheken
  - Verwendung und Konfiguration zusätzlicher Plugins
  - Angabe von Repositories
- `pom.xml`-Datei wird auf Wohlgeformtheit und Gültigkeit zu Beginn jedes Maven-Aufrufs geprüft

Project Object Model – pom.xml II

- Standard-Konfigurationsdatei

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
              http://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>de.dhbwka.java.exercise</groupId>

    <artifactId>my-maven-project</artifactId>

    <version>0.0.1-SNAPSHOT</version>

    <name>Maven Demo Project</name>

    <packaging>jar</packaging>
</project>
```

Gruppenzugehörigkeit, meist Java-Package

Projekt-Identifier

Aktuelle Versionsnummer

Name des Projekts

Typ des resultierenden Software-Artefakts

maven

pom.xml Abhängigkeitsverwaltung

- Abhängigkeiten („Dependencies“) verwalten
  - Google GSON via Maven einbinden

```
<project xmlns="..." xmlns:xsi="..." xsi:schemaLocation="...">

(…)

   <dependencies>
      <dependency>
         <groupId>com.google.code.gson</groupId>
         <artifactId>gson</artifactId>
         <version>2.11.0</version>
      </dependency>
   </dependencies>

(…)

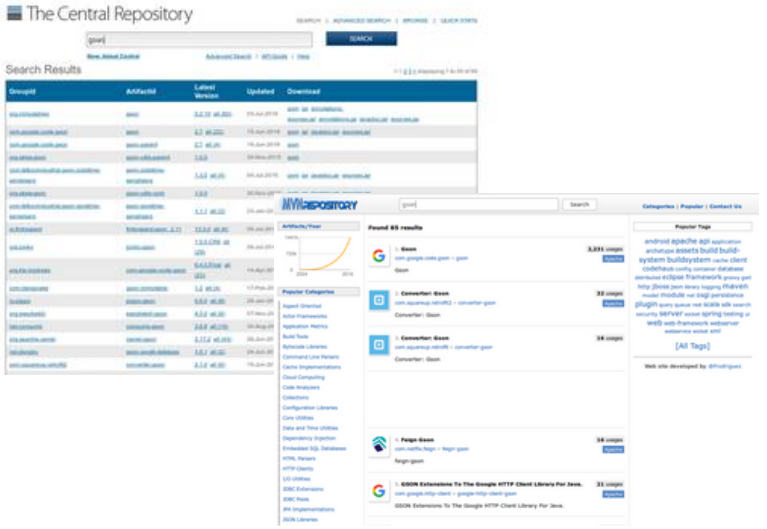
</project>
```

maven

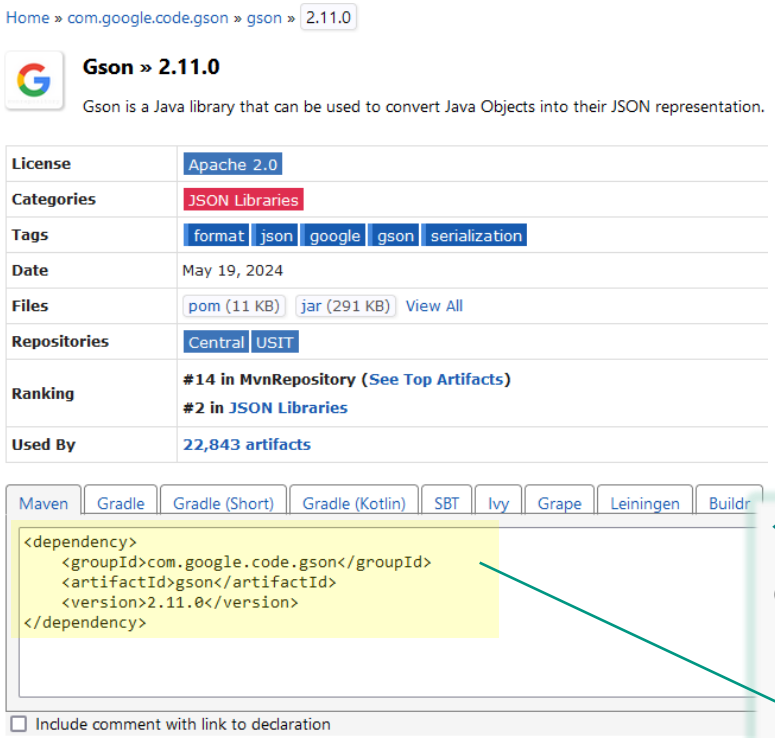
`pom.xml` Abhängigkeitsverwaltung II

- Gibt es ein Verzeichnis verfügbarer Bibliotheken?
  - Bibliotheken aus Standard-Repository immer verfügbar
  - Fremd-Repositories einbindbar

- Übersicht über Standard-Repository
  - http://search.maven.org/
    Standard-Suchmaschine
    für zentrales Maven-Repo
  - http://mvnrepository.com/
    Alternative Suchmaschine
    für zentrales Maven-Repo



pom.xml Abhängigkeitsverwaltung III




```xml
<project xmlns="..." xmlns:xsi="..." xsi:schemaLocation="...">

(…)

   <dependencies>
      <dependency>
         <groupId>com.google.code.gson</groupId>
         <artifactId>gson</artifactId>
         <version>2.11.0</version>
      </dependency>
   </dependencies>

(…)

</project>
```

maven

Maven Plugins

- Maven bringt eine Reihe von plugins mit, die über die Kommandozeile genutzt werden können
- Beispiele:
  - mvn clean: Löscht die unter target bereits gespeicherten compilierten .class-Dateien
  - mvn compiler: Compiliert Quellcodedateien
- Darüber hinaus besitzt jedes plugin sogenannte "goals", welche die konkrete Funktionalität referenzieren
- Diese können über das plugin mit einem Doppelpunkt angesprochen werden
  - Beispiel: mvn compiler:compile → Startet den Compilierprozess

Maven Plugins (2)

- Häufig genutzte goals wie compile besitzen meist shortcuts
  - mvn compiler:compile ⇔ mvn compile
  - mvn clean:clean ⇔ mvn clean
- Plugins können über Einträge in der pom.xml konfiguriert werden

```
<build>
   <plugins>
      <plugin>
         <groupId>org.apache.maven.plugins</groupId>
         <artifactId>maven-compiler-plugin</artifactId>
         <configuration>
            (…)
         </configuration>
      </plugin>
   </plugins>
</build>
```

maven

Maven Plugins (3)

- Beispiel-Konfiguration für das maven-compiler-plugin

```
<configuration>
   <release>25</release>
</configuration>
```

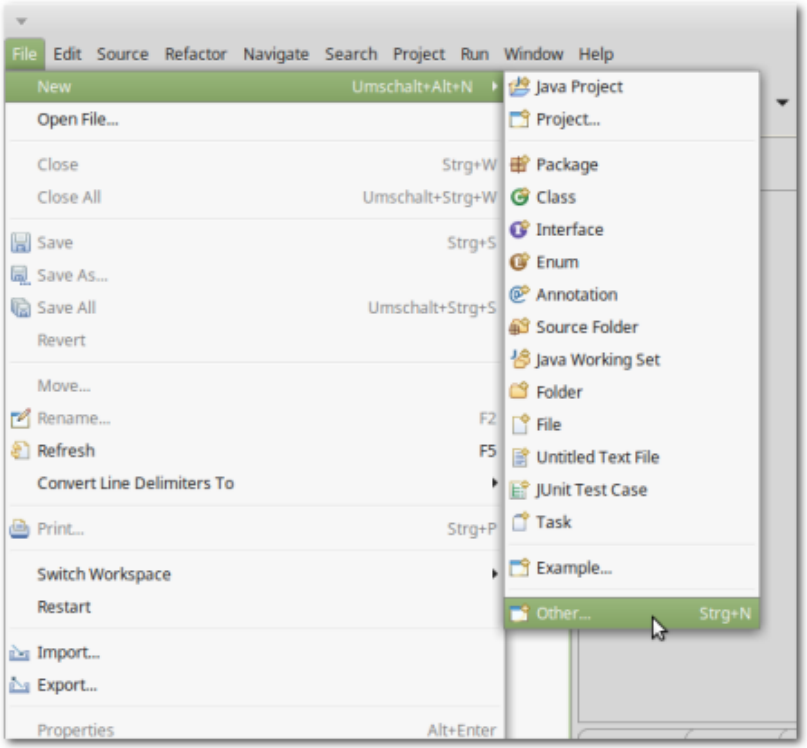
- Konfiguration kann aber auch über Kommandozeilenbefehl vorgenommen werden
  - `mvn clean compile -Dmaven.compiler.release=25`
  - manche Konfigurationen werden nicht über Kommandozeilenparameter unterstützt

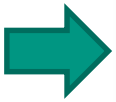
Maven-Projekt erstellen

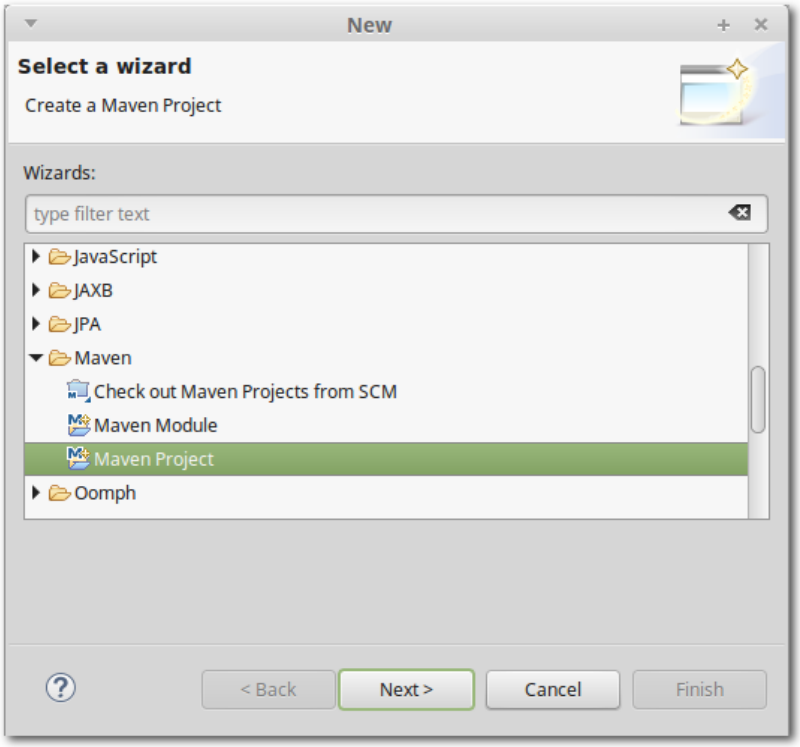
- Grundsätzlich: Maven ist ein Kommandozeilenwerkzeug
  - Projekt kann über Kommandozeile erstellt werden
  - Breite, UI-unterstützte Integration in IDEs

- Neue Projekte können basierend auf Projektschablonen („Archetypes“) erstellt werden
  - Wir nutzen zunächst eine „nackte“ Schablone

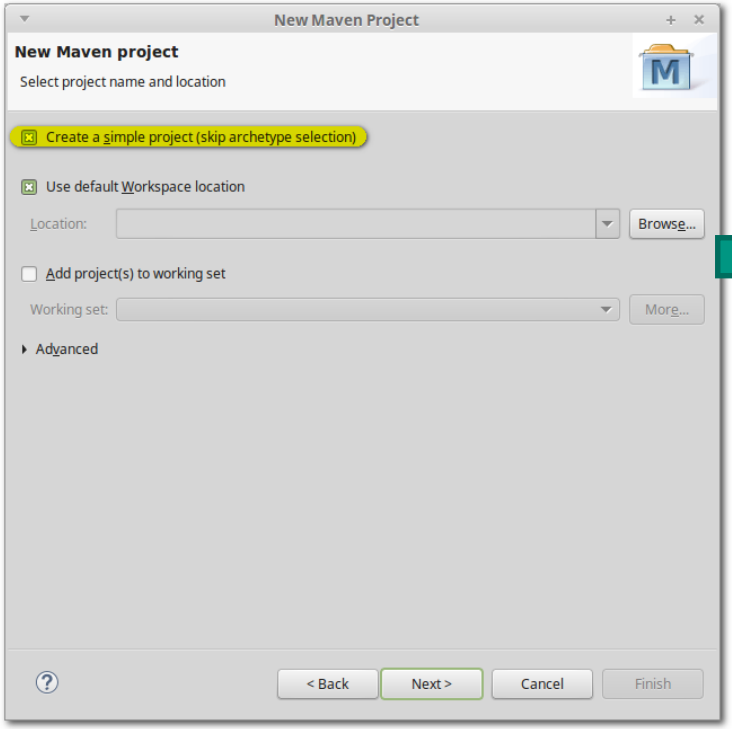
Maven-Projekt erstellen: Eclipse

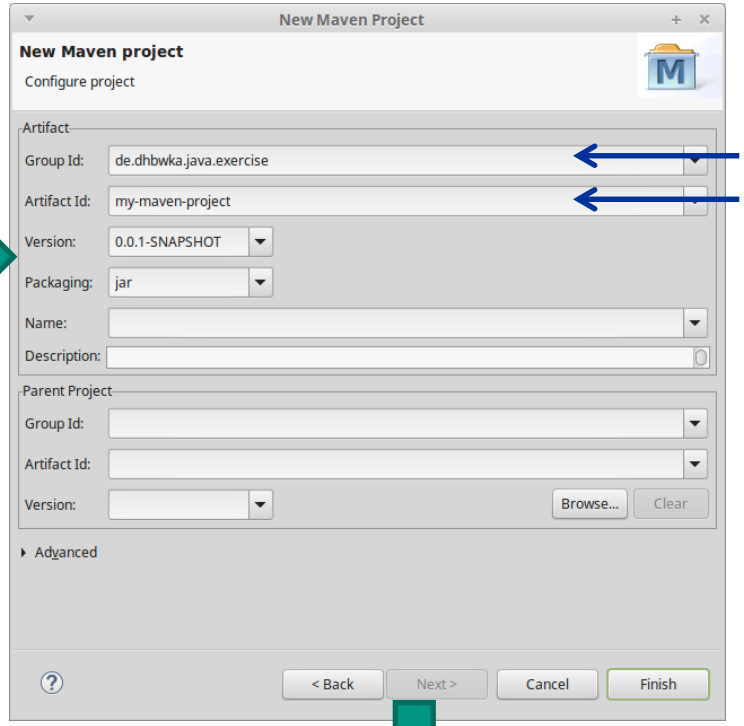






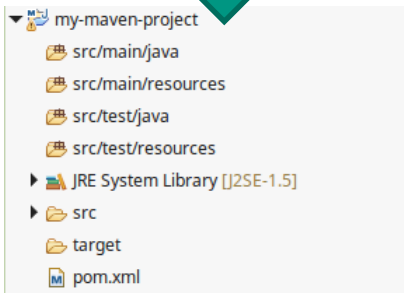
Maven-Projekt erstellen: Eclipse II



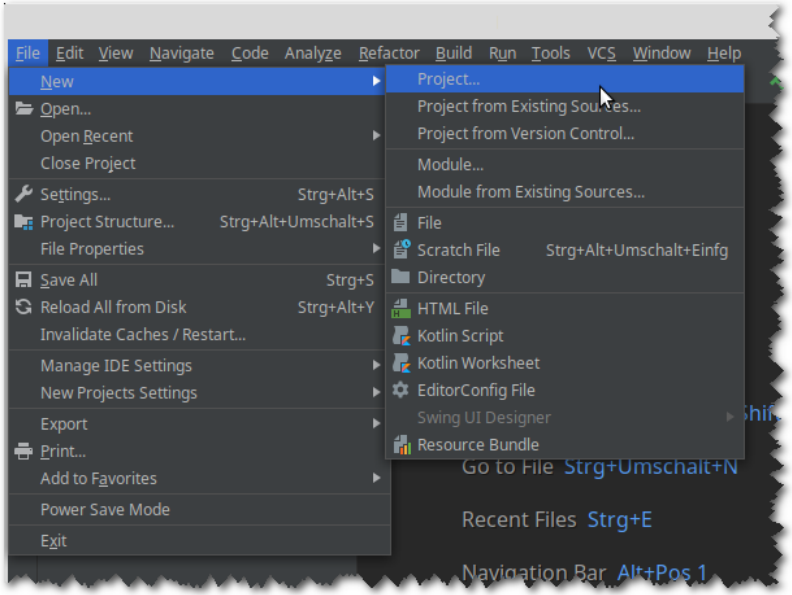


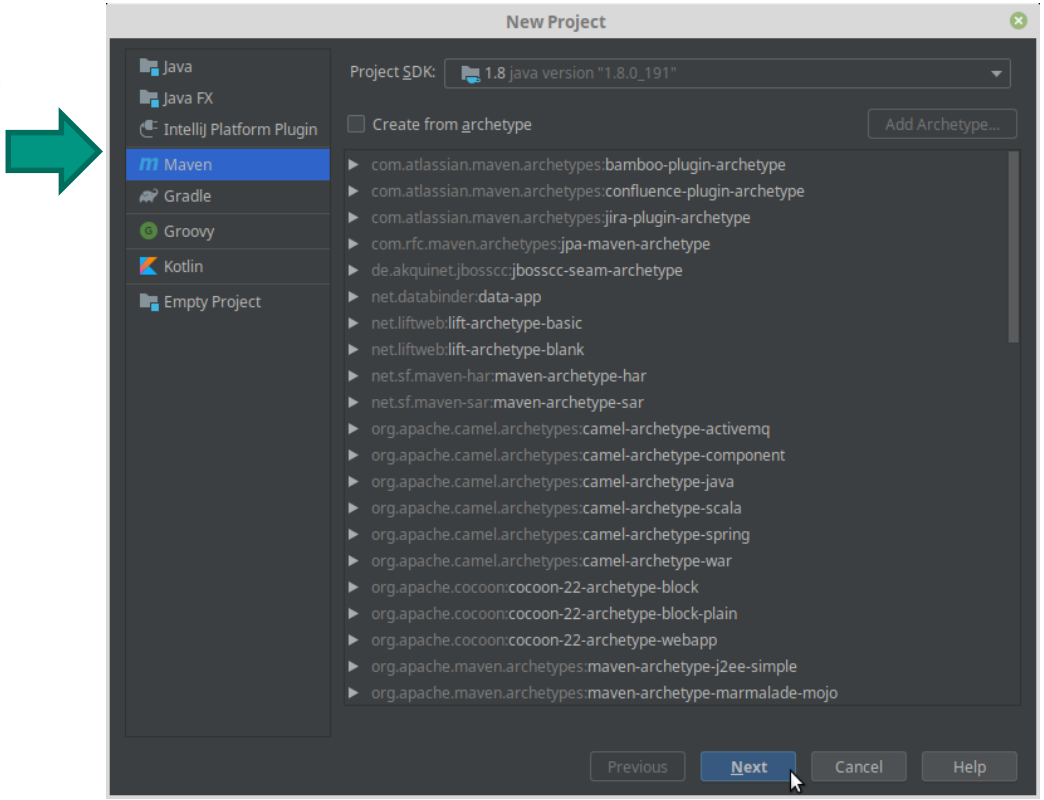
Projektgruppe

Projekt-ID



Maven-Projekt erstellen: IntelliJ

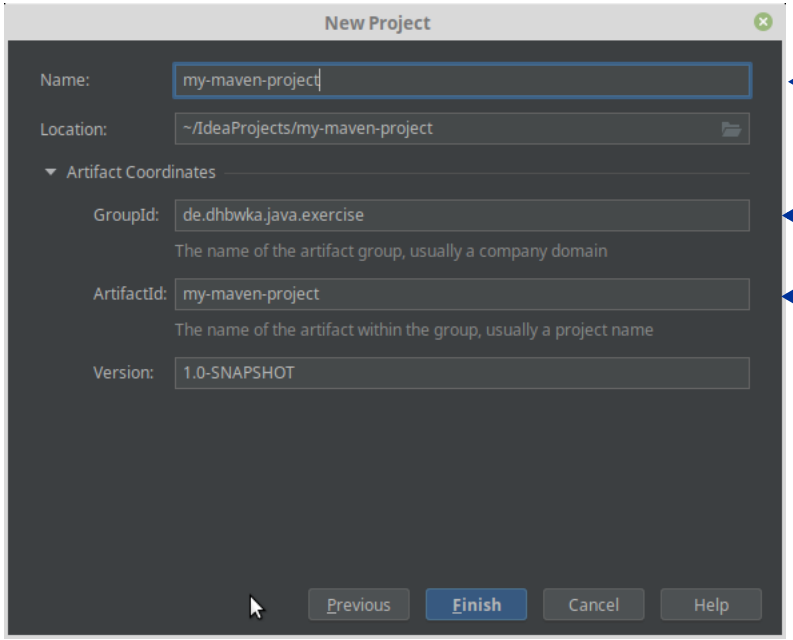




IntelliJ IDEA

maven

Maven-Projekt erstellen: IntelliJ II

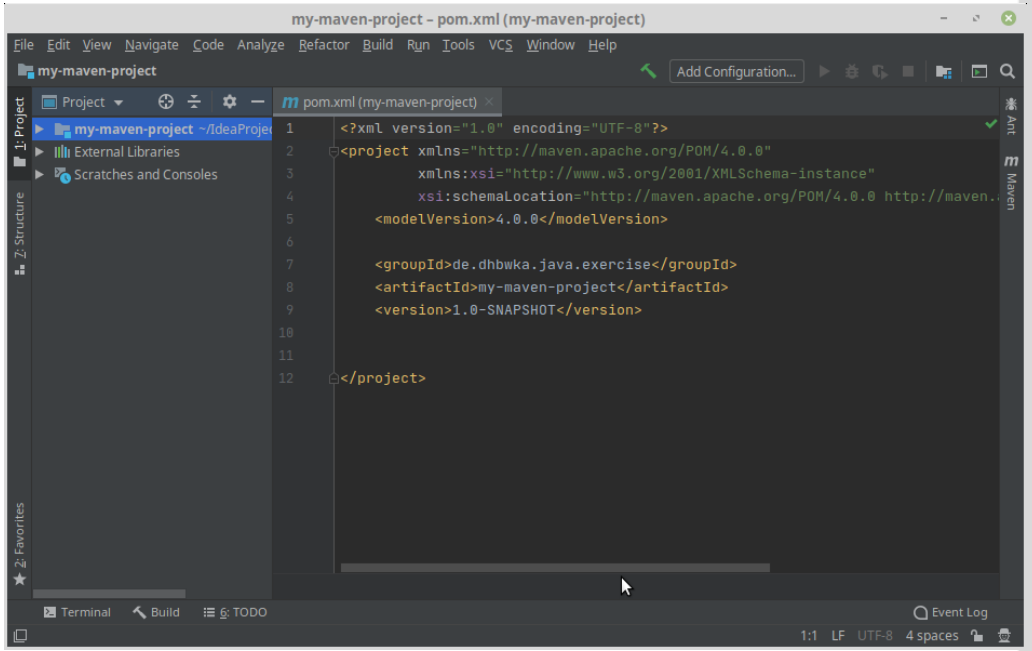


IntelliJ-Projektname

Projektgruppe

Maven-Projekt-ID

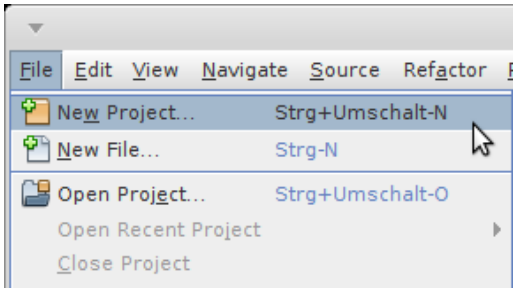
kann identisch sein, muss nicht



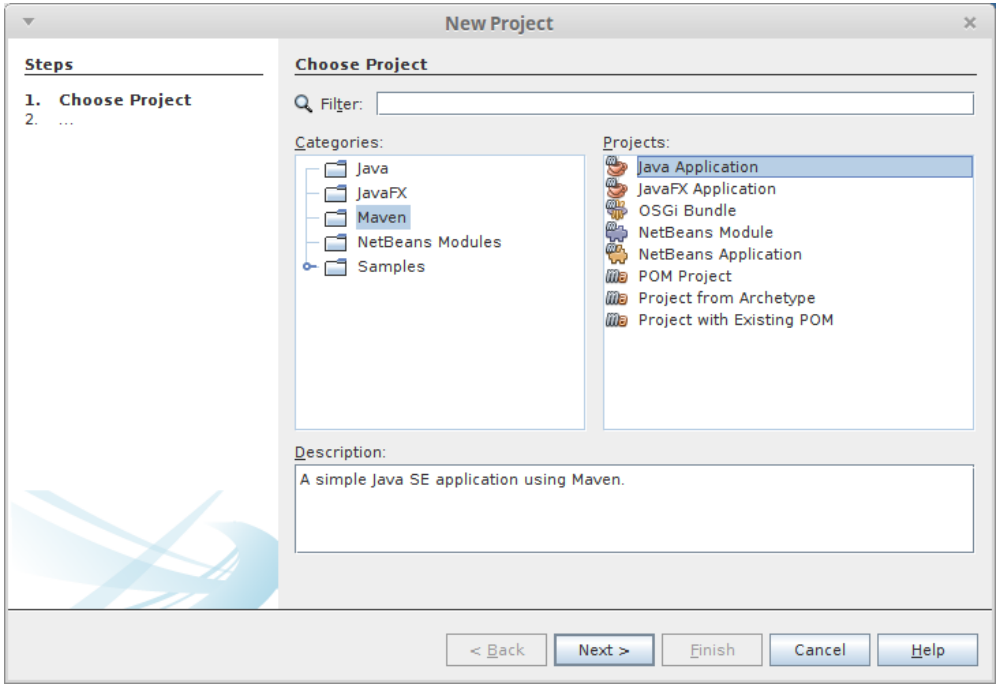
IntelliJ IDEA

maven

Maven-Projekt erstellen: NetBeans



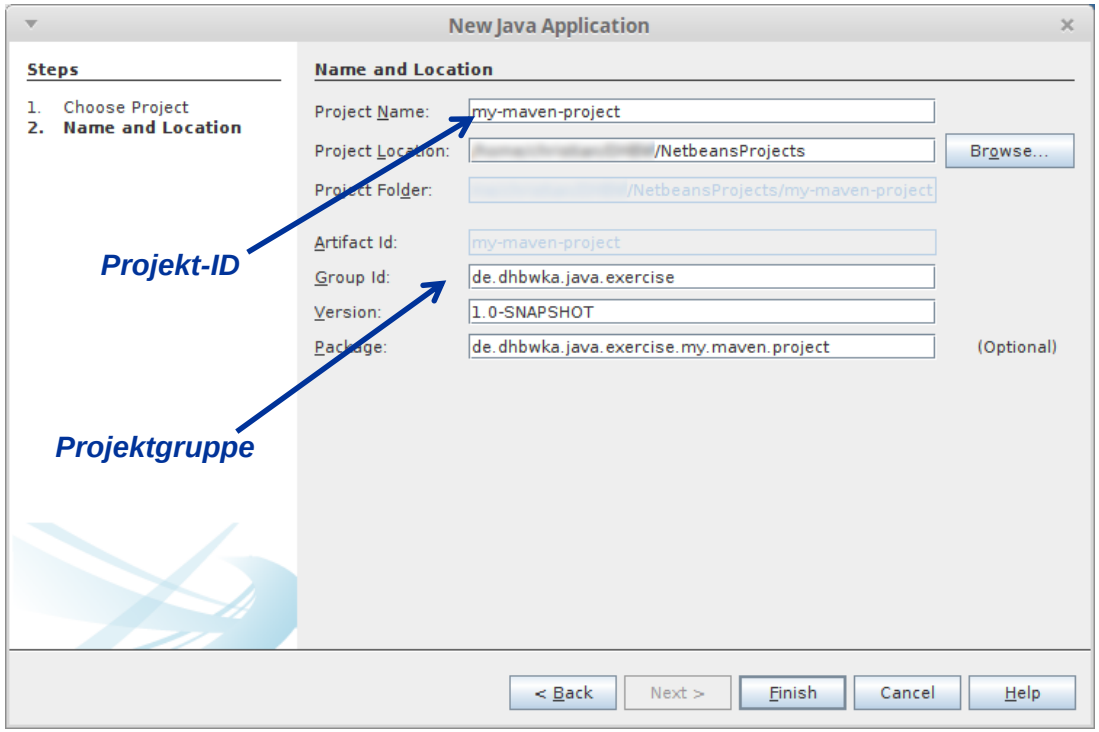


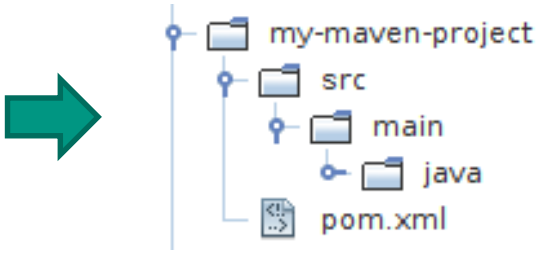


NetBeans

maven

Maven-Projekt erstellen: NetBeans II





Aufgabe – Erstes Maven-Projekt

- Erstellen Sie ein Maven-Projekt
  - Abhängigkeit zu Google GSON
    - https://mvnrepository.com/artifact/com.google.code.gson/gson
  - Konfigurieren der Java-Version auf Java 21 (oder höher) über das maven-compiler-plugin
  - Encoding auf UTF-8 für das maven-compiler-plugin einstellen